Einführung: Algorithmen, Übersetzung und das erste Programm

PROG · KAPITEL 01 · SPICKZETTEL

ALGORITHMUS

Eine genaue, endliche Schrittfolge, die ein Problem löst. Jeder Schritt ist ohne Rückfrage ausführbar.

Anforderungen: **Endlichkeit**, **Ausführbarkeit**, **Korrektheit**, **Effizienz**.

Begriff	Bedeutung
terminierend	endet nach endlich vielen Schritten
determiniert	gleiche Eingabe → gleiches **Ergebnis**
deterministisch	nächster Schritt immer eindeutig → gleicher **Weg**

Merksatz: Deterministisch ist immer auch determiniert, umgekehrt nicht.

DATENSTRUKTUREN

Struktur	Idee	Beispiel
Array	gleicher Typ, hintereinander, Index	Matrizen
Liste	Elemente verweisen aufeinander	dynamische Sammlungen
Baum	Knoten mit Unterknoten	Dateisystem
Graph	Knoten und Kanten	Routenplanung
Stack	LIFO: Letztes zuerst raus	Zurück-Taste
Queue	FIFO: Erstes zuerst raus	Druckaufträge, Rechnungen nach Eingang

In Python eingebaut: `list`, `tuple`, `dict`, `set`.

PARADIGMEN

Paradigma	Idee	Beispiel
imperativ/prozedural	Schritt für Schritt	C
objektorientiert	Objekte mit Daten und Methoden	Java
funktional	Funktionen ohne Seiteneffekte	Haskell
deklarativ/logisch	beschreiben, was erreicht werden soll	SQL

Python ist **multiparadigmatisch**.

VOM QUELLCODE ZUM PROGRAMM

Variante	Ablauf	Beispiele
Interpreter	Anweisung für Anweisung sofort ausführen	Shell, PHP
Compiler	vorab alles in Maschinencode	C, C++, Rust, Go
Zwischencode	Bytecode, eine VM führt ihn aus	Java, C#, Python

- Compiler: schnell, aber vor jedem Test kompilieren, nur eine Zielplattform.
- JIT: VM übersetzt oft genutzten Code zur Laufzeit in Maschinencode.
- Python (CPython): Quellcode → Bytecode → Python-VM.

WERKZEUGE

Werkzeug	Aufgabe	Aufruf
uv	Python, Projekte, Pakete, starten	`uv run hallo.py`
ruff	formatieren, Fehler finden	`uv run ruff check .`
ty	Datentypen prüfen	`uv run ty check`

Terminal: `pwd`, `ls` / `dir`, `cd ordner`, `cd ..`. Editor: Tab = 4 Leerzeichen, UTF-8.

DAS ERSTE PROGRAMM

```
# Kommentar
print("Hello World!")  # Hello World!
```

- Einrückung bildet Blöcke (4 Leerzeichen), Doppelpunkt leitet sie ein.
- Kein Semikolon, eine Zeile = eine Anweisung.
- Groß-/Kleinschreibung zählt: `Print` ≠ `print`.
- Konsole (`>>>`) zeigt Werte auch ohne `print`, Text mit Anführungszeichen: `'Hallo'`. In einer `.py`-Datei brauchst du `print()`.

SYNTAXFEHLER ODER LAUFZEITFEHLER?

Beim Start prüft CPython zuerst die **Syntax der ganzen Datei**, dann läuft sie Zeile für Zeile.

```
print("A")
Print("B")   # NameError in Zeile 2
print("C")
# Ausgabe: A, dann Fehler
```

```
print("A")
print("B"    # SyntaxError
print("C")
# Ausgabe: nur Fehler, kein "A"
```

FEHLERMELDUNG LESEN

1. Letzte Zeile: **was**? (z. B. `NameError`)
2. Zeile darüber: **wo**? (Datei, Zeile)
3. Hinweis lesen: `Did you mean: 'print'?`

TYPISCHE FEHLER

- Determiniert (Ergebnis) und deterministisch (Weg) verwechselt.
- LIFO und FIFO vertauscht.
- „Python ist eine reine Interpretersprache“: falsch, Python nutzt Zwischencode.
- Nicht ausführbarer Schritt wie „Rechnung prüfen“, kein Fall für Abweichungen.
- `Print` statt `print`, fehlendes Anführungszeichen oder Klammer.
- In der `.py`-Datei Ausdruck ohne `print()` und Ausgabe erwartet.
- Tabs und Leerzeichen gemischt.